Empresa Ejemplo S.A.
Gerencia de Arquitectura y Desarrollo de Software

Estándar de Desarrollo de Aplicaciones Web

Angular, ASP.NET Core y SQL Server

Código	Versión	Fecha de aprobación	Estado
EST-WEB-001	3.0	2026-02-02	Vigente

Aprobado por: Comité de Arquitectura

Documento de prueba con contenido ficticio. La empresa y las reglas son un ejemplo para pruebas de un asistente RAG; no corresponden a ninguna organización real.

Control de cambios

Versión	Fecha	Descripción del cambio
1.0	2020-07-01	Versión inicial para ASP.NET MVC y AngularJS.
2.0	2023-04-17	Migración a ASP.NET Core y Angular. Se incorpora la nomenclatura de base de datos.
3.0	2026-02-02	Nueva nomenclatura de base de datos (tablas en singular, esquemas por módulo, prefijo usp_). Componentes standalone en Angular. .NET 8 LTS como mínimo.

Contenido

Control de cambios . . . 2
1. Objetivo . . . 3
2. Alcance . . . 3
3. Definiciones y niveles de cumplimiento . . . 3
4. Arquitectura y estructura de la solución . . . 3
5. Convenciones de código C# (backend) . . . 4
6. Convenciones Angular (frontend) . . . 6
7. Base de datos SQL Server: nomenclatura . . . 7
8. Base de datos SQL Server: programación T-SQL y acceso a datos . . . 9
9. Pruebas y calidad . . . 10
10. Relación con otros estándares . . . 11

1. Objetivo

Unificar la arquitectura, las convenciones de código y la nomenclatura de base de datos de las aplicaciones web de la empresa, para que cualquier desarrollador pueda mantener cualquier sistema y para que la revisión de código sea objetiva.

2. Alcance

Aplica a las aplicaciones web nuevas y a los cambios sobre las existentes construidas con Angular (frontend), ASP.NET Core (backend) y SQL Server (base de datos). La nomenclatura de base de datos de este documento aplica también a toda base de datos nueva, sea cual sea la aplicación que la consuma. Las reglas de seguridad están en EST-SEG-001 y las de diseño de APIs en EST-API-001.

3. Definiciones y niveles de cumplimiento

Término	Definición
Obligatorio	La regla debe cumplirse en todo desarrollo nuevo y en todo cambio sobre código existente. Su incumplimiento bloquea el pase a producción salvo excepción aprobada.
Recomendado	Buena práctica esperada. Su incumplimiento no bloquea el pase, pero debe justificarse en la revisión de código.
Prohibido	Práctica que no debe aparecer en el código. Equivale a una regla obligatoria en sentido negativo.
Excepción	Autorización temporal (máximo 6 meses) para no cumplir una regla obligatoria. La aprueba el Comité de Arquitectura y, si la regla es de seguridad, también Seguridad de la Información. Se registra en el formulario FOR-ARQ-07.
SPA	Single Page Application: aplicación Angular que se ejecuta en el navegador y consume APIs.
PascalCase	Cada palabra inicia en mayúscula, sin separadores: DetallePedido.
camelCase	Como PascalCase pero la primera letra en minúscula: detallePedido.
kebab-case	Palabras en minúscula separadas por guion: detalle-pedido.

4. Arquitectura y estructura de la solución

4.1 WEB-ARQ-001 · Arquitectura de referencia

Nivel: Obligatorio | **Aplica a:** Web

Las aplicaciones web se construyen como una SPA en Angular que consume APIs REST en ASP.NET Core, las que acceden a SQL Server. La solución .NET se organiza en capas siguiendo Clean Architecture: `<Sistema>.Api` (controladores y configuración), `<Sistema>.Application` (casos de uso, DTOs, validaciones), `<Sistema>.Domain` (entidades y reglas de negocio, sin dependencias externas) e `<Sistema>.Infrastructure` (EF Core, Dapper, integraciones). La capa Domain no referencia a ninguna otra capa.

4.2 WEB-ARQ-002 · Versiones de plataforma soportadas

Nivel: Obligatorio | **Aplica a:** Web | **Referencias:** EST-SEG-001 SEG-CFG-003

Desarrollos nuevos: .NET 8 LTS o una versión LTS posterior con soporte vigente, Angular en una versión con soporte activo (18 o superior) y SQL Server 2019 o 2022. Está prohibido iniciar desarrollos nuevos en .NET Framework. Los sistemas existentes deben migrar a una versión con soporte antes de que termine el soporte de la versión que usan.

4.3 WEB-ARQ-003 · Configuración por entorno

Nivel: Obligatorio | **Aplica a:** Web | **Referencias:** EST-SEG-001 SEG-CRI-003

La configuración se separa por entorno (`appsettings.Development.json`, `appsettings.Production.json` y `environment.ts` / `environment.prod.ts` en Angular). Los archivos de configuración no contienen secretos (ver SEG-CRI-003). Las URLs de APIs no se escriben en el código de los servicios: se leen de la configuración.

4.4 WEB-ARQ-004 · Repositorio y ramas

Nivel: Obligatorio | **Aplica a:** Web

Cada sistema tiene un repositorio en Azure DevOps con la estructura `src/`, `tests/`, `db/` y `docs/`. Se usa trunk-based development con ramas cortas `feature/<ticket>-<descripcion>` y `hotfix/<ticket>`. La rama `main` está protegida: solo se integra por pull request.

5. Convenciones de código C# (backend)

5.1 WEB-NET-001 · Nomenclatura en C#

Nivel: Obligatorio | **Aplica a:** Web, API

PascalCase para clases, records, métodos, propiedades, constantes y espacios de nombres; camelCase para variables locales y parámetros; `_camelCase` para campos privados; prefijo `I` para interfaces; sufijo `Async` para métodos asíncronos. Los nombres se escriben en español para conceptos de negocio (Cliente, Pedido) y se permiten términos técnicos en inglés (Repository, Handler). Están prohibidas las abreviaturas no estándar y la notación húngara.

Ejemplo incorrecto

```
public class cliente_service
{
    private readonly IClienteRepository repo;
    public async Task<ClienteDto> obtener(int ID) { ... }
    string strNombre;
}
```

Ejemplo correcto

```
public class ClienteService : IClienteService
{
    private readonly IClienteRepository _clienteRepository;
    public async Task<ClienteDto?> ObtenerAsync(int clienteId, CancellationToken ct) { ... }
}
```

5.2 WEB-NET-002 · Asincronía de extremo a extremo

Nivel: Obligatorio | **Aplica a:** Web, API

Toda operación de entrada/salida (base de datos, HTTP, archivos) es asíncrona y se espera con `await`. Está prohibido bloquear con `.Result`, `.Wait()` o `GetAwaiter().GetResult()` y declarar métodos `async void` (salvo manejadores de eventos). Los métodos públicos asíncronos reciben y propagan un `CancellationToken`.

Ejemplo incorrecto

```
public ClienteDto Obtener(int id)
{
    var cliente = _contexto.Clientes.FindAsync(id).Result;
    return _mapper.Map<ClienteDto>(cliente);
}
```

Ejemplo correcto

```
public async Task<ClienteDto?> ObtenerAsync(int id, CancellationToken ct)
{
    var cliente = await _contexto.Clientes.FindAsync([id], ct);
    return cliente is null ? null : _mapper.Map<ClienteDto>(cliente);
}
```

5.3 WEB-NET-003 · Inyección de dependencias

Nivel: Obligatorio | **Aplica a:** Web, API

Las dependencias se reciben por constructor y se registran en el contenedor de ASP.NET Core con el tiempo de vida adecuado (Scoped para servicios con DbContext). Está prohibido instanciar servicios con `new` dentro de otras clases, usar `IServiceProvider` como localizador de servicios y mantener estado mutable en servicios Singleton.

5.4 WEB-NET-004 · Manejo de excepciones

Nivel: Obligatorio | **Aplica a:** Web, API | **Referencias:** EST-SEG-001 SEG-LOG-003

Las excepciones no controladas se capturan en un middleware global (`UseExceptionHandler` con `IExceptionHandler`) que registra el error y devuelve Problem Details (ver API-RES-002). Está prohibido capturar `Exception` para ignorarla (bloque `catch` vacío) y usar excepciones para el control de flujo normal. Los errores de negocio esperados se modelan con excepciones de dominio propias o con un tipo resultado.

Ejemplo incorrecto

```
try { await _pedidos.RegistrarAsync(pedido, ct); }
catch (Exception) { }
```

Ejemplo correcto

```
try { await _pedidos.RegistrarAsync(pedido, ct); }
catch (StockInsuficienteException ex)
{
    _logger.LogWarning(ex, "Stock insuficiente para el pedido {PedidoId}", pedido.Id);
    throw;
}
```

5.5 WEB-NET-005 · Logging estructurado

Nivel: Obligatorio | **Aplica a:** Web, API | **Referencias:** EST-SEG-001 SEG-LOG-001

Se usa `ILogger<T>` con plantillas de mensaje y propiedades con nombre; el proveedor corporativo es Serilog con salida a Application Insights. Está prohibido construir el mensaje con interpolación de cadenas o concatenación, y usar `Console.WriteLine` para registrar eventos.

Ejemplo incorrecto

```
_logger.LogInformation($"Pedido {pedido.Id} registrado por {usuario}");
```

Ejemplo correcto

```
_logger.LogInformation("Pedido {PedidoId} registrado por {Usuario}", pedido.Id, usuario);
```

5.6 WEB-NET-006 · Calidad del compilado

Nivel: Obligatorio | **Aplica a:** Web, API

Los proyectos tienen habilitados `<Nullable>enable</Nullable>`, `<AnalysisLevel>latest-recommended</AnalysisLevel>` y `<TreatWarningsAsErrors>true</TreatWarningsAsErrors>` en la configuración Release. Las supresiones de advertencias se justifican con un comentario en el código.

5.7 WEB-NET-007 · Tamaño de métodos y clases

Nivel: Recomendado | **Aplica a:** Web, API

Un método no debería superar las 40 líneas ni tener una complejidad ciclomática mayor a 10; una clase no debería superar las 400 líneas. Superar estos límites requiere justificación en la revisión de código.

6. Convenciones Angular (frontend)

6.1 WEB-ANG-001 · Estructura del proyecto Angular

Nivel: Obligatorio | **Aplica a:** Web

El proyecto se organiza por funcionalidad: `core/` (servicios singleton, interceptores, guards), `shared/` (componentes, pipes y directivas reutilizables) y `features/<funcionalidad>/` (componentes, servicios y rutas de cada módulo de negocio). Los componentes nuevos son standalone.

6.2 WEB-ANG-002 · Nomenclatura en Angular

Nivel: Obligatorio | **Aplica a:** Web

Archivos en kebab-case con sufijo de tipo: `cliente-lista.component.ts`, `cliente.service.ts`, `auth.guard.ts`, `cliente.model.ts`. Clases en PascalCase con sufijo (`ClienteListaComponent`, `ClienteService`). Selectores de componentes con el prefijo corporativo `emp-` (`emp-cliente-lista`). Observables con sufijo `$` (`clientes$`).

Ejemplo incorrecto

```
// archivo: ClientesList.ts
@Component({ selector: 'clientesList', ... })
export class clientesList { clientes: any; }
```

Ejemplo correcto

```
// archivo: cliente-lista.component.ts
@Component({ selector: 'emp-cliente-lista', standalone: true, ... })
export class ClienteListaComponent { clientes$!: Observable<Cliente[]>; }
```

6.3 WEB-ANG-003 · Tipado estricto

Nivel: Obligatorio | **Aplica a:** Web

El archivo `tsconfig.json` tiene `"strict": true` y las opciones estrictas de plantillas de Angular habilitadas. Está prohibido el tipo `any`; si el tipo es desconocido se usa `unknown` con una verificación de tipo. Los contratos con las APIs se modelan con interfaces en archivos `*.model.ts`.

Ejemplo incorrecto

```
obtener(id: any): Observable<any> {
  return this.http.get('/api/v1/clientes/' + id);
}
```

Ejemplo correcto

```
obtener(clienteId: number): Observable<Cliente> {
  return this.http.get<Cliente>(`${this.baseUrl}/clientes/${clienteId}`);
}
```

6.4 WEB-ANG-004 · Acceso HTTP solo desde servicios

Nivel: Obligatorio | **Aplica a:** Web

Los componentes no inyectan `HttpClient`: las llamadas HTTP se encapsulan en servicios de cada funcionalidad. Las suscripciones se manejan con el pipe `async` o con `takeUntilDestroyed()`; está prohibido dejar suscripciones manuales sin liberar.

Ejemplo incorrecto

```
export class ClienteListaComponent implements OnInit {
  constructor(private http: HttpClient) {}
  ngOnInit() { this.http.get<Cliente[]>('/api/v1/clientes').subscribe(c => this.clientes = c); }
}
```

Ejemplo correcto

```
export class ClienteListaComponent {
  private readonly clienteService = inject(ClienteService);
  readonly clientes$ = this.clienteService.listar();   // plantilla: clientes$ | async
}
```

6.5 WEB-ANG-005 · Interceptores y almacenamiento de tokens

Nivel: Obligatorio | **Aplica a:** Web | **Referencias:** EST-SEG-001 SEG-AUT-005

La autenticación se integra con MSAL Angular o con un backend-for-frontend (BFF) que maneja la cookie de sesión. Los encabezados de autenticación y el manejo global de errores HTTP se implementan en interceptores funcionales. Está prohibido guardar tokens en `localStorage` o `sessionStorage` (ver SEG-AUT-005).

6.6 WEB-ANG-006 · Seguridad en plantillas

Nivel: Obligatorio | **Aplica a:** Web | **Referencias:** EST-SEG-001 SEG-INY-003

Se confía en la sanitización automática de Angular. Está prohibido usar los métodos `bypassSecurityTrust*` con datos externos, manipular el DOM directamente con `ElementRef.nativeElement.innerHTML` y construir URLs de navegación con datos no validados (ver SEG-INY-003).

6.7 WEB-ANG-007 · Formularios

Nivel: Obligatorio | **Aplica a:** Web | **Referencias:** EST-SEG-001 SEG-INY-002

Se usan formularios reactivos tipados (`FormGroup` con tipos) con las validaciones declaradas en el componente. Los mensajes de validación se muestran junto al campo. Las validaciones del frontend no reemplazan a las del backend (ver SEG-INY-002).

6.8 WEB-ANG-008 · Rendimiento en Angular

Nivel: Recomendado | **Aplica a:** Web

Las rutas de cada funcionalidad se cargan de forma diferida (`loadComponent` / `loadChildren`), los componentes usan `ChangeDetectionStrategy.OnPush` y las listas usan `track` en `@for`.

7. Base de datos SQL Server: nomenclatura

Esta nomenclatura es obligatoria para toda base de datos nueva desde la versión 3.0 de este estándar. Las bases existentes que usan la nomenclatura anterior (prefijo TBL_ y mayúsculas, ver EST-DSK-001) no se renombran: los objetos nuevos que se agreguen a ellas siguen la nomenclatura de la base.

7.1 WEB-BD-001 · Esquemas por módulo

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API

Los objetos se agrupan en esquemas por módulo de negocio en minúsculas (`ventas`, `seguridad`, `maestro`, `auditoria`). Está prohibido crear objetos nuevos en el esquema `dbo`.

7.2 WEB-BD-002 · Nombres de tablas

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API

Las tablas se nombran en PascalCase, en singular y sin prefijos: `ventas.Cliente`, `ventas.DetallePedido`. Está prohibido usar prefijos como `tbl`, `TBL_` o `T_`, espacios, guiones o palabras reservadas. Las tablas de relación muchos a muchos combinan ambos nombres: `seguridad.UsuarioRol`.

Ejemplo incorrecto

```
CREATE TABLE dbo.TBL_CLIENTES ( ... );
CREATE TABLE dbo.tblDetallePedidos ( ... );
```

Ejemplo correcto

```
CREATE TABLE ventas.Cliente ( ... );
CREATE TABLE ventas.DetallePedido ( ... );
```

7.3 WEB-BD-003 · Nombres de columnas y claves

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API

Las columnas se nombran en PascalCase. La clave primaria se llama `<Tabla>Id` (`ClienteId`) y es `INT IDENTITY` o `BIGINT IDENTITY` según el volumen esperado. Las claves foráneas tienen el mismo nombre que la clave primaria que referencian. Las columnas booleanas inician con `Es` o `Tiene` (`EsActivo`, `TieneDeuda`).

7.4 WEB-BD-004 · Nombres de restricciones e índices

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API

Restricciones con nombre explícito: `PK_<Tabla>`, `FK_<TablaHija>_<TablaPadre>`, `UQ_<Tabla>_<Columna>`, `CK_<Tabla>_<Columna>`, `DF_<Tabla>_<Columna>`. Índices: `IX_<Tabla>_<Columna1>_<Columna2>`. Está prohibido dejar que SQL Server genere nombres automáticos.

Ejemplo incorrecto

```
CREATE TABLE ventas.Pedido (
    PedidoId INT IDENTITY PRIMARY KEY,
    ClienteId INT REFERENCES ventas.Cliente(ClienteId)
);
```

Ejemplo correcto

```
CREATE TABLE ventas.Pedido (
    PedidoId  INT IDENTITY(1,1) NOT NULL,
    ClienteId INT NOT NULL,
    CONSTRAINT PK_Pedido PRIMARY KEY (PedidoId),
    CONSTRAINT FK_Pedido_Cliente FOREIGN KEY (ClienteId) REFERENCES ventas.Cliente(ClienteId)
);
CREATE INDEX IX_Pedido_ClienteId ON ventas.Pedido (ClienteId);
```

7.5 WEB-BD-005 · Nombres de procedimientos, vistas y funciones

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API

Procedimientos almacenados: `usp_<Entidad>_<Accion>` (`ventas.usp_Cliente_Listar`, `ventas.usp_Pedido_Registrar`). Vistas: `vw_<Nombre>`. Funciones: `fn_<Nombre>`. Disparadores: `tr_<Tabla>_<Evento>`. Está prohibido el prefijo `sp_`, reservado para procedimientos del sistema (SQL Server los busca primero en la base master).

7.6 WEB-BD-006 · Tipos de datos

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API

Texto: `NVARCHAR(n)` con longitud definida; `NVARCHAR(MAX)` solo para textos largos justificados. Montos: `DECIMAL(18,2)` (o `DECIMAL(18,4)` para tasas). Fechas y horas: `DATETIME2(3)` en UTC; `DATE` cuando no se necesita la hora. Booleanos: `BIT`. Está prohibido usar `FLOAT` o `REAL` para montos, `MONEY`, y los tipos obsoletos `TEXT`, `NTEXT` e `IMAGE`.

7.7 WEB-BD-007 · Columnas de auditoría y borrado lógico

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API

Toda tabla transaccional incluye `FechaCreacion DATETIME2(3)`, `UsuarioCreacion NVARCHAR(100)`, `FechaModificacion DATETIME2(3) NULL` y `UsuarioModificacion NVARCHAR(100) NULL`. Los registros maestros se desactivan con `EsActivo BIT` en lugar de eliminarse físicamente.

8. Base de datos SQL Server: programación T-SQL y acceso a datos

8.1 WEB-BD-008 · Estructura de procedimientos almacenados

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API

Los procedimientos inician con `SET NOCOUNT ON;` y `SET XACT_ABORT ON;`. Los que modifican datos usan transacción explícita dentro de `BEGIN TRY ... END TRY BEGIN CATCH ... END CATCH`, hacen `ROLLBACK` si hay una transacción abierta y relanzan el error con `THROW`.

Ejemplo correcto

```
CREATE OR ALTER PROCEDURE ventas.usp_Pedido_Registrar
    @ClienteId INT, @Total DECIMAL(18,2), @Usuario NVARCHAR(100), @PedidoId INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        INSERT INTO ventas.Pedido (ClienteId, Total, FechaCreacion, UsuarioCreacion)
        VALUES (@ClienteId, @Total, SYSUTCDATETIME(), @Usuario);
        SET @PedidoId = SCOPE_IDENTITY();
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END
```

8.2 WEB-BD-009 · Consultas explícitas

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API

Está prohibido `SELECT *` en procedimientos, vistas y código de aplicación: se listan las columnas. Las consultas sobre tablas grandes filtran por columnas indexadas y evitan funciones sobre columnas en el `WHERE` (por ejemplo `YEAR(Fecha) = 2026`), que impiden usar índices.

Ejemplo incorrecto

```
SELECT * FROM ventas.Pedido WHERE YEAR(FechaCreacion) = 2026;
```

Ejemplo correcto

```
SELECT PedidoId, ClienteId, Total, FechaCreacion
FROM ventas.Pedido
WHERE FechaCreacion >= '2026-01-01' AND FechaCreacion < '2027-01-01';
```

8.3 WEB-BD-010 · Acceso a datos desde .NET

Nivel: Obligatorio | **Aplica a:** Web, API | **Referencias:** EST-SEG-001 SEG-INY-001

Se usa Entity Framework Core para operaciones CRUD y Dapper para consultas de lectura complejas o de alto rendimiento. Toda consulta es parametrizada (ver SEG-INY-001): con EF Core se permite `FromSql` / `FromSqlInterpolated` y está prohibido `FromSqlRaw` o `ExecuteSqlRaw` con cadenas concatenadas. Las consultas de solo lectura usan `AsNoTracking()`.

Ejemplo incorrecto

```
var clientes = await _contexto.Clientes
    .FromSqlRaw("SELECT * FROM ventas.Cliente WHERE Nombre LIKE '%" + filtro + "%'")
    .ToListAsync(ct);
```

Ejemplo correcto

```
var patron = $"%{filtro}%";
var clientes = await _contexto.Clientes
    .FromSql($"SELECT ClienteId, Nombre, Documento FROM ventas.Cliente WHERE Nombre LIKE {patron}")
    .AsNoTracking()
    .ToListAsync(ct);
```

8.4 WEB-BD-011 · Versionado de cambios de base de datos

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API

Todo cambio de esquema se versiona en el repositorio (carpeta `db/`) como migración de EF Core o script de DbUp, es idempotente y se despliega por el pipeline. Está prohibido aplicar cambios manuales en los ambientes de certificación y producción.

8.5 WEB-BD-012 · Usuario de base de datos de la aplicación

Nivel: Obligatorio | **Aplica a:** Web, Escritorio, API | **Referencias:** EST-SEG-001 SEG-GEN-002

Cada aplicación usa su propio usuario o identidad administrada con permisos `EXECUTE` y `SELECT` limitados a sus esquemas. Está prohibido conectarse con `sa`, con usuarios del rol `db_owner` o con cuentas personales (ver SEG-GEN-002).

9. Pruebas y calidad

9.1 WEB-QA-001 · Pruebas unitarias

Nivel: Obligatorio | **Aplica a:** Web, API

Las capas Application y Domain tienen una cobertura mínima de 70% con xUnit. En Angular se prueban los servicios y los componentes con lógica de presentación. Las pruebas no dependen de bases de datos ni servicios externos reales.

9.2 WEB-QA-002 · Nombres de pruebas

Nivel: Obligatorio | **Aplica a:** Web, API

Los métodos de prueba siguen el formato `Metodo_Escenario_ResultadoEsperado` (`RegistrarAsync_ClienteInactivo_LanzaExcepcion`) y la estructura Arrange / Act / Assert.

9.3 WEB-QA-003 · Pull requests y quality gate

Nivel: Obligatorio | **Aplica a:** Web, API | **Referencias:** EST-SEG-001 SEG-CFG-004

Todo pull request requiere la aprobación de al menos un revisor distinto del autor y un pipeline exitoso: compilación, pruebas, quality gate de SonarQube sin issues nuevos de severidad Blocker o Critical, análisis de dependencias y detección de secretos.

10. Relación con otros estándares

En caso de conflicto entre este documento y EST-SEG-001 prevalece EST-SEG-001. En caso de conflicto con EST-DSK-001 en nomenclatura de base de datos prevalece este documento para bases de datos nuevas, por ser la versión más reciente.